

# 01

## StockRush

분산 커머스 / 주문  
상태 수렴 / 이벤트  
기반 백엔드

### ROLE

개인 프로젝트 / 설계,  
구현, 검증

### SKILLS

분산 트랜잭션  
이벤트 수렴  
Gateway 보안  
장애 복구

### STACK

Spring Boot

Kafka

PostgreSQL

Keycloak

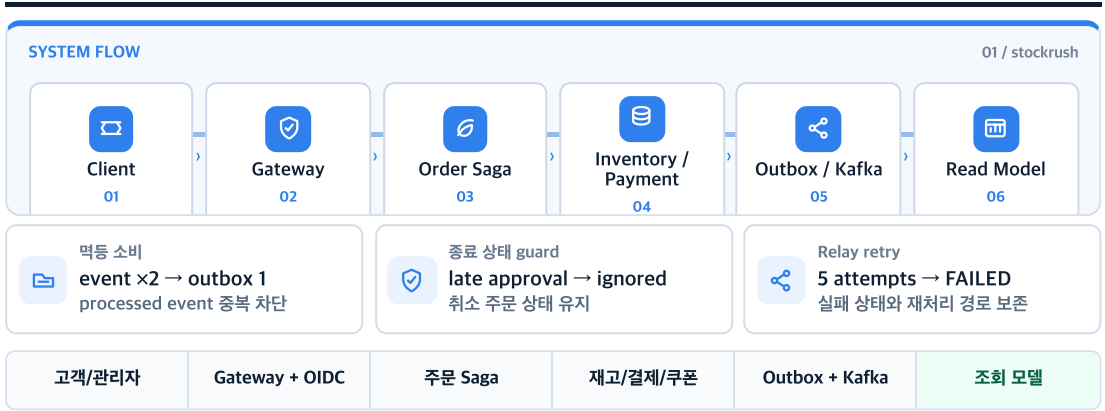
Docker Compose

Android smoke

### PROJECT 01 · 분산 커머스

## 부분 실패 뒤에도 주문 상태를 수렴시키는 이벤트 기반 커머스

한정 재고 주문의 부분 실패와 중복 처리를 Saga, Transactional Outbox, 멍든 소비자로 제어하고 주문·재고·결제·조회 모델의 최종 상태 수렴을 장애 시나리오로 검증했습니다.



#### 01 해결한 문제

부분 실패와 재시도 뒤 주문·재고·결제·조회 상태가 어긋나는 문제  
동시 주문, 결제 실패와 지연, Kafka 중단, 이벤트 재처리 상황에서는 주문 상태와 재고 수량, 결제 결과, 조회 모델이 서로 다른 속도로 바뀝니다. 서비스 경계와 복구 흐름을 코드와 시나리오로 분리해 최종 상태 수렴을 검증했습니다.

#### 02 핵심 설계 판단

| 01                                                          | 02                                                              | 03                                                          |
|-------------------------------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------|
| <b>Saga orchestration</b><br>재고·결제·쿠폰·출고 상태 전이를 주문 서비스에서 조율 | <b>Reliable publish</b><br>DB 변경과 outbox row를 함께 저장해 발행 실패를 재처리 | <b>Trust boundary</b><br>Gateway JWT와 서비스 내부 리소스 소유권 검사를 분리 |

#### 03 대표 코드 근거

##### PersistentCreateOrderService.create

.../application/PersistentCreateOrderService.java

```
boolean saved = orderRepository.saveIfAbsent(result.order(),
command.idempotencyKey());
if (!saved) {
    return
    CreateOrderResult.replayed(replayOrder(command.idempotencyKey(),
command.memberId()));
}
outboxEventRepository.save(result.outboxEvent());
```

주문 저장과 outbox 기록을 같은 트랜잭션에 두고 멍든 충돌은 기존 주문 재조회로 수렴시킵니다.

TEST PersistentCreateOrderServiceIntegrationTest

##### OrderSagaEventHandler.transitionOrder

.../order/application/OrderSagaEventHandler.java

```
where order_id = :orderId
and status not in ('CANCELLED', 'CONFIRMED')

return updated == 1;
```

종료 상태 주문의 추가 전이를 SQL에서 차단해 늦은 이벤트가 취소·확정 주문을 되살리지 못하게 합니다.

TEST OrderSagaEventHandlerIntegrationTest

##### OutboxRelayService.claimPending

.../infra/outbox/OutboxRelayService.java

```
where status = 'PENDING'
and (next_retry_at is null or next_retry_at <= now())
order by created_at, id
limit :batchSize
for update skip locked
```

재시도 가능 이벤트만 잠금 선점해 여러 relay의 중복 발행을 피하고 소진 실패를 FAILED로 남깁니다.

TEST OutboxRelayServiceIntegrationTest

#### 04 검증 결과

- OrderSagaEventHandlerIntegrationTest가 동일 이벤트 2회 처리에도 outbox 1건만 남도록 고정합니다.
- 취소 완료 뒤 늦게 도착한 결제 승인 이벤트가 주문을 되살리지 않는 상태 guard를 검증합니다.
- OutboxRelayServiceIntegrationTest가 retry 5회 소진 뒤 FAILED 전이와 재처리 상태를 확인합니다.

#### OUTCOME

한정 판매 커머스의 동시 주문, 결제 실패/지연, 이벤트 발행 실패, 중복 처리, 운영자 보정을 재현 가능한 흐름으로 묶었습니다.

#### VERIFIED SCOPE

코드와 통합 테스트로 멍든 소비·종료 상태 guard·relay retry를 다뤘으며 실패·다중 리전·운영 규모 부하는 포함하지 않았습니다.